

0.93.1. Un jeu de bombes — le O bouge, les X l'arrêtent

Le 0.93, et le O bouge avec la croix, un pas toutes les 150 ms. Avant chaque pas, lire() regarde la case d'arrivée : un X, et le O reste où il est.

1. Ce qu'il faut comprendre

C'est le 0.93, avec une chose en plus : le O bouge. La croix le déplace d'une case, dans les couloirs. Les X (le cadre et les piliers) l'arrêtent.

Deux variables pour le O : `ox` et `oy`, sa colonne et sa ligne. Il part de (1, 1). Et deux autres, `nx` et `ny` : la case d'arrivée, là où il irait s'il bougeait. C'est la méthode du mur M (le 0.82) : **on calcule d'abord, on regarde, et seulement après on bouge.**

chaque(150) (le 0.77) : au plus un pas toutes les 150 ms, un peu plus vite que le serpent.

Étape 1, la case d'arrivée. On part de la case actuelle (`nx = ox; ny = oy;`), puis la flèche tenue change `nx` ou `ny` d'une case. **Ce qui est nouveau ici :** `else if`. `if (...) { ... } else if (...) { ... }` : « sinon, si... ». Dès qu'une flèche est trouvée, les suivantes ne sont **pas** regardées. Avec DROITE et BAS tenues ensemble, seule DROITE compte : **pas de pas en diagonale** (en diagonale, le O passerait entre deux piliers). `else` seul, on l'a vu au 0.83.1.

Étape 2 : une flèche est-elle tenue ? Si aucune ne l'est, l'arrivée est la case où l'on est déjà : `nx != ox || ny != oy` est faux, on ne fait rien.

Étape 3 : regarder l'arrivée. `lire(nx, ny) != ALPHABET[23]` (le 0.82) : « la case d'arrivée n'est pas un X ». Alors on efface le O, on change `ox` et `oy`, et on le pose sur sa nouvelle case.

Déroulons : O en (1, 1), DROITE : l'arrivée est (2, 1). `lire(2, 1)` : un espace, pas un X. Le O va en (2, 1). Puis BAS : l'arrivée est (2, 2), un **pilier** (colonne et ligne paires, le 0.93). `lire(2, 2)` rend le X : le O ne bouge pas. Il faut revenir en colonne 1 ou aller en colonne 3 pour descendre.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **677 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l’a demandé — et ce qu’il a laissé, parce que rien ne l’appelle :

Gravé dans la cartouche :

démarrage	63 o	← la console, à l’allumage	– la mise en route de la console
chaque_minuteur()	124 o	← main()	– une fonction fournie par le langage
main()	281 o	← démarrage	– ta fonction
VBlank	8 o	← la console, à chaque image	– compte les images, 60 fois par seconde
AttendreImage	28 o	← image()	– attend l’image suivante
AttendreVBlank	16 o	← démarrage	– attend que l’écran soit libre
AttendreAcces	12 o	← EffacerCases	– attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), lire(), effacer()	– trouve l’adresse d’une case de l’écran
EffacerCases	12 o	← effacer()	– efface des cases
CopierTuiles	40 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l’écran
LireManette	37 o	← bouton()	– lit les huit boutons
Tuiles	32 o	← CopierTuiles	– 2 lettres sur 43

Lettres gravées : OX

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

poser()	× 4	52 o sur place, dans main()	+ AdresseCase (10 o)
bouton()	× 4	20 o sur place, dans main()	+ LireManette (37 o)
effacer()	× 1	15 o sur place, dans main()	+ AdresseCase (10 o), EffacerCases (12 o)
lire()	× 1	11 o sur place, dans main()	+ AdresseCase (10 o)
image()	× 1	3 o sur place, dans main()	+ AttendreImage (28 o)
images()	× 1	2 o sur place, dans chaque_minuteur()	

Tes fonctions appelées :

chaque_minuteur()	× 1	10 o sur place, dans main()
-------------------	-----	-----------------------------

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- UN JEU DE BOMBES ----
// Les lettres : X = un mur, O = le joueur, B = une bombe, W = un ennemi.
// ALPHABET[23] : le X   ALPHABET[14] : le O
// ALPHABET[1]  : le B   ALPHABET[22] : le W

#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
#include <chaque>     // répond 1 toutes les n millisecondes
#include <bouton>     // lit un bouton de la manette
#include <lire>       // lit la tuile posée sur une case
#include <effacer>    // efface des cases, ou tout le fond
```

2. UNE VARIABLE — OX

```
uint8_t ox = 1;          // NOUVEAU : le O, le joueur : sa colonne...
```

3. UNE VARIABLE — OY

```
uint8_t oy = 1;          // ...et sa ligne. Il part du coin (1, 1).
```

4. UNE VARIABLE — NX

```
uint8_t nx = 0;          // la case d'ARRIVÉE, calculée AVANT de bouger (le 0.82)
```

5. UNE VARIABLE — NY

```
uint8_t ny = 0;
```

6. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // ---- Le terrain : le cadre et les piliers X (le 0.93) ----
    for (uint8_t l = 0; l < 17; l++) {
        for (uint8_t c = 0; c < 19; c++) {
            if (l == 0 || l == 16 || c == 0 || c == 18) {    // le cadre
                poser(c, l, ALPHABET[23]);
            }
            if (c % 2 == 0 && l % 2 == 0) {                  // les piliers : colonne ET ligne paires
                poser(c, l, ALPHABET[23]);
            }
        }
    }

    poser(ox, oy, ALPHABET[14]);                          // le 0 à sa place (le 0.93)

    while (true) {
        image();

        // ---- NOUVEAU : le 0 bouge avec la croix, un pas toutes les 150 ms ----
        if (chaque(150)) {
            // 1. la case d'ARRIVÉE : on part de la case actuelle (le 0.82).
            //     else if : une seule flèche à la fois, pas de pas en diagonale.
            nx = ox;
            ny = oy;
            if (bouton(DROITE)) {
                nx = ox + 1;
            } else if (bouton(GAUCHE)) {
                nx = ox - 1;
            } else if (bouton(BAS)) {
                ny = oy + 1;
            } else if (bouton(HAUT)) {
                ny = oy - 1;
            }

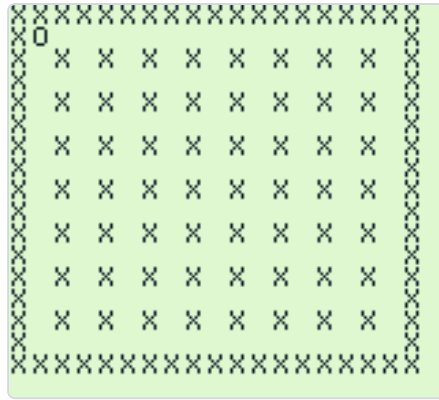
            // 2. une flèche est tenue (l'arrivée n'est pas la case où l'on est)...
            if (nx != ox || ny != oy) {
                // 3. ...et l'arrivée n'est pas un X : le 0 y va.
                if (lire(nx, ny) != ALPHABET[23]) {
                    effacer(ox, oy, 1);                    // le 0 quitte sa case...
                    ox = nx;                                // ...et va sur la case d'arrivée
                    oy = ny;
                    poser(ox, oy, ALPHABET[14]);
                }
            }
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



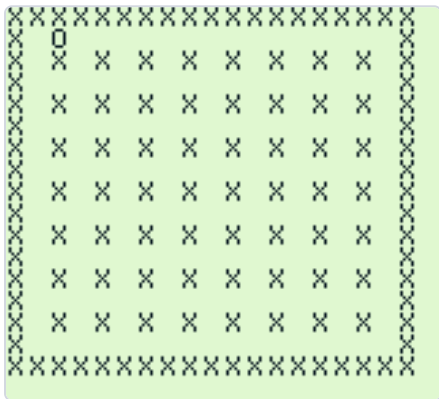
la toute première image — la console vient de commencer — image 1



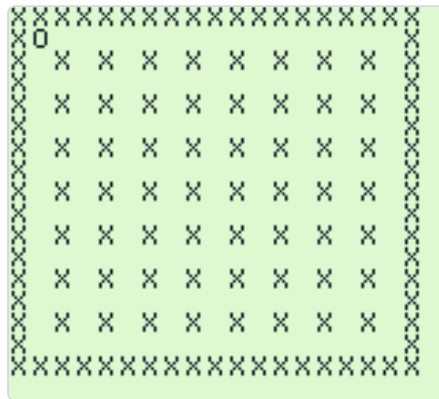
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

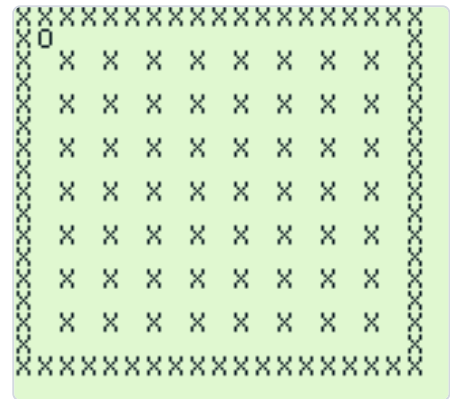
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



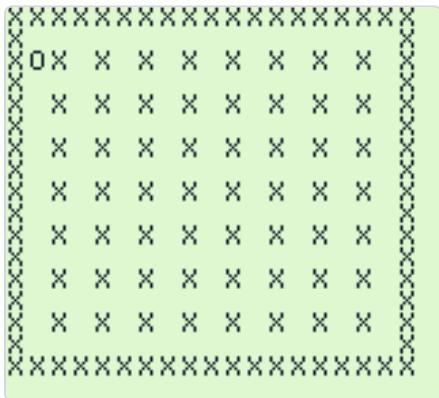
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran n'a pas bougé



BAS — l'écran a changé

5. Ce qu'on doit voir

Le O se déplace avec la croix dans les couloirs ; il ne traverse ni le cadre, ni les piliers.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ à droite : le O va en (2, 1)
- ✓ en bas, le pilier (2, 2) l'arrête (oy = 1)
- ✓ en haut, le cadre l'arrête (oy = 1)

Ensuite : **0.93.2. Un jeu de bombes — A pose une bombe** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++